ЛАБОРАТОРНА РОБОТА № 3

Тема: Управляючі конструкції в мові Go. Функції. Введення та виведення інформації у консоль

Мета: Ознайомитися з управляючими конструкціямимовипрограмування Go, навчитися користуватися функціями вводу/виводутастворювати користувацькі функції. Ознайомитись з методамигенеруваннявипадкових чисел.

хід роботи

Завдання 1.

Розробити функцію генерування цілочислової послідовностіпсевдовипадкових значень (за допомогою конгруентного методу) тавиконатиобробку отриманого масиву даних наступним чином:

Таблиця 3.2 – Вхідні данні

Варіант	Коефіцієнти			Діапазон випадкових величин, n	Довжина послідовності чисел, K
	a	c	m		
1	1103515245	12345	2^{31}	[0, 100)	10000
2	1664525	1013904223	2^{32}	[0, 150	20000
3	16807	0	2^{31}-1	[0, 200)	30000
4	22695477	1	2^{32}	[0, 250)	40000
5	48271	0	2^{31}-1	[0, 300)	50000
6	214013	2531011	2^{32}	[0, 100)	50000
7	2147483629	2147483587	2^{31}-1	[0, 150)	40000
8	65539	0	2^{32}	[0, 200)	30000
9	1140671485	12820163	2^{24}	[0, 250)	20000
10	69069	1	2^{32}	[0, 300)	10000
11	1103515245	12345	2^{31}	[0, 300)	50000
12	1664525	1013904223	2^{32}	[0, 250)	40000
13	16807	0	2^{31}-1	[0, 200)	30000
14	22695477	1	2^{32}	[0, 150)	20000
15	48271	0	2^{31}-1	[0, 100)	10000
16	214013	2531011	2^{32}	[0, 300)	10000
17	2147483629	2147483587	2^{31}-1	[0, 250)	20000
18	65539	0	2^{32}	[0, 200)	30000
19	1140671485	12820163	2^{24}	[0, 150)	40000
20	134775813	1	2^{32}	[0, 100)	50000

Рис. 3.1

Виконання завдання:

randIntSeq генерує цілочислову послідовність псевдовипадкових значень лінійним конгруентним методом: Xk+1 = (a*Xk + c) mod m. Повертає K значень у діапазоні [0, n).

Код програми:

```
func nextInt(seed *int64, a, c, m int64) int64 {
   next := (a*(*seed) + c) % m
   if next < 0 {
      next += m
   }
   *seed = next
   return next
}
```

Результат виконання:

```
=== Лабораторна робота №3 ===
Лінійний конгруентний метод: X_{k+1} = (a*X_k + c) mod m

Таблиця 3.2 — варіанти (1-20):
  Варіант |    a         |      c       |   m     | Діапазон  |    K
  --------|--------------|--------------|---------|-----------|-------
        1 |   1103515245 |        12345 | 2^31    | [0, 100  ) | 10000
        2 |      1664525 |   1013904223 | 2^32    | [0, 150  ) | 20000
        3 |        16807 |            0 | 2^31-1  | [0, 200  ) | 30000
        4 |     22695477 |            1 | 2^32    | [0, 250  ) | 40000
        5 |        48271 |            0 | 2^31-1  | [0, 300  ) | 50000
        6 |       214013 |      2531011 | 2^32    | [0, 100  ) | 50000
        7 |   2147483629 |   2147483587 | 2^31-1  | [0, 150  ) | 40000
        8 |        65539 |            0 | 2^32    | [0, 200  ) | 30000
        9 |   1140671485 |     12820163 | 2^24    | [0, 250  ) | 20000
       10 |        69069 |            1 | 2^32    | [0, 300  ) | 10000
       11 |   1103515245 |        12345 | 2^31    | [0, 300  ) | 50000
       12 |      1664525 |   1013904223 | 2^32    | [0, 250  ) | 40000
       13 |        16807 |            0 | 2^31-1  | [0, 200  ) | 30000
       14 |     22695477 |            1 | 2^32    | [0, 150  ) | 20000
       15 |        48271 |            0 | 2^31-1  | [0, 100  ) | 10000
       16 |       214013 |      2531011 | 2^32    | [0, 300  ) | 10000
       17 |   2147483629 |   2147483587 | 2^31-1  | [0, 250  ) | 20000
       18 |        65539 |            0 | 2^32    | [0, 200  ) | 30000
       19 |   1140671485 |     12820163 | 2^24    | [0, 150  ) | 40000
       20 |    134775813 |            1 | 2^32    | [0, 100  ) | 50000

Введіть номер варіанту (1-20):
```

Рис. 3.2

Завдання 2.

Обробити згенеровану послідовність псевдовипадкових чисел:
визначити частоти появи значень, обчислити статистичні ймовірності,
знайти математичне сподівання, дисперсію та середньоквадратичне відхилення.

Виконання завдання:

Код програми:

1. Частоти

```
fmt.Printf("1) Частота інтервалів (перші %d значень 0..%d):\n", show, show-1)
for i := 0; i < show; i++ {
    fmt.Printf("   значення %d: частота = %d\n", i, freq[i])
}
if n > show {
    fmt.Println("   ...")
```

2. Ймовірності

```
prob := make([]float64, n)
for i := 0; i < n; i++ {
    prob[i] = float64(freq[i]) / float64(v.K)
}
fmt.Printf("\n2) Статистична ймовірність P_i = частота_i / K (перші 5):\n")
for i := 0; i < 5 && i < n; i++ {
    fmt.Printf("   P(%d) = %.6f\n", i, prob[i])
}
```

3. Математичне сподівання

```
var mx float64
for i := 0; i < n; i++ {
    mx += float64(i) * prob[i]
}
fmt.Printf("\n3) Математичне сподівання M(X) = %.6f\n", mx)

var dx float64
for i := 0; i < n; i++ {
    diff := float64(i) - mx
    dx += prob[i] * (diff * diff)
}
```

4. Дисперсія

```
fmt.Printf("4) Дисперсія D(X) = %.6f\n", dx)
```

5. Виклики в main()

```
sigma := math.Sqrt(dx)
fmt.Printf("5) Середньоквадратичне відхилення sigma = %.6f\n", sigma)
```

Результат виконання:

```
--- Завдання 1: обробка цілочисельної послідовності ---
1) Частота інтервалів (перші 10 значень 0..9):
   значення 0: частота = 162
   значення 1: частота = 157
   значення 2: частота = 139
   значення 3: частота = 140
   значення 4: частота = 163
   значення 5: частота = 145
   значення 6: частота = 128
   значення 7: частота = 160
   значення 8: частота = 158
   значення 9: частота = 152
   ...

2) Статистична ймовірність P_i = частота_i / K (перші 5):
   P(0) = 0.005400
   P(1) = 0.005233
   P(2) = 0.004633
   P(3) = 0.004667
   P(4) = 0.005433
```

Рис. 3.3

Завдання 3.

Розробити функцію генерування послідовності псевдовипадкових дійсних значень (за допомогою конгруентного методу) (таблиця 3.2). Отримати K значень у діапазоні [0, n).

Виконання завдання:

Код програми:

Функція генерації дійсних чисел

```
fmt.Printf("Перші %d значень у діапазоні [0, %d):\n", showReal, v.N)
  for i := 0; i < showReal; i++ {
    fmt.Printf("   [%d] = %.4f\n", i, realSeq[i])
  }
```

```
    seed3 := x0
    realLong := generateRealSequence(&seed3, v)
    var sum float64
    for _, val := range realLong {
      sum += val
    }
    meanReal := sum / float64(len(realLong))
    var sumSq float64
    for _, val := range realLong {
      d := val - meanReal
      sumSq += d * d
    }
    varReal := sumSq / float64(len(realLong))
    fmt.Printf("\nДля K=%d дійсних значень: середнє = %.4f, дисперсія = %.4f\n", v.K, meanReal,
varReal)
}
```

Результат виконання:

```
Перші 10 значень у діапазоні [0, 200):
   [0] = 0.0016
   [1] = 26.3076
   [2] = 151.1211
   [3] = 91.7300
   [4] = 106.5534
   [5] = 43.7918
   [6] = 9.4089
   [7] = 135.7729
   [8] = 135.8593
   [9] = 186.9386

Для K=30000 дійсних значень: середнє = 99.9657, дисперсія = 3312.4264

Process finished with the exit code 0
```

Рис. 3.4

Висновок

У ході лабораторної роботи було опановано керуючі конструкції мови **Go** та механізми взаємодії з користувачем через функції введення-виведення. Практична частина роботи полягала у розробці **лінійного конгруентного генератора (LCG)** для створення послідовностей псевдовипадкових чисел.

На основі отриманих даних проведено повний статистичний аналіз, що включав розрахунок частотних характеристик, ймовірностей, а також ключових показників: **математичного сподівання, дисперсії та середньоквадратичного відхилення**. Додатково було реалізовано алгоритм генерації дійсних чисел, що дозволило розширити функціонал розробленого генератора.

Контрольні питання

1. Які керуючі конструкції існують в мові Go?

У Go використовуються три основні конструкції для керування потоком виконання:

- **if** — умовний перехід.
- **for** — єдина конструкція для організації циклів.
- **switch** — багатоканальне розгалуження (вибір одного з багатьох варіантів).

2. Як організувати цикли?

Усі види циклів (класичний лічильник, ітерація по колекції або аналог while) реалізуються за допомогою ключового слова **for**. Go свідомо не має окремих ключових слів while або do-while.

3. Призначення for

Вона призначена для багаторазового повторення блоку коду. Вона може працювати як:

Класичний цикл: for i := 0; i < 10; i++ { ... }

Умовний цикл (аналог while): for condition { ... }

Нескінченний цикл: for { ... }

Ітератор (з range): для обходу масивів, слайсів та карт.

4. Призначення if

if використовується для виконання коду лише у випадку, якщо задана логічна умова є істинною (true). Особливістю Go є можливість короткого оголошення змінної безпосередньо в рядку з умовою (наприклад, if err := someFunc(); err != nil { ... }).

5. Призначення switch

switch спрощує написання складних розгалужень, замінюючи довгі ланцюжки if-else. У Go switch автоматично зупиняється після виконання обраного варіанта (не потребує break), але дозволяє переходити далі за допомогою ключового слова fallthrough.

6. Як здійснюється форматований вивід даних?

Для форматованого виводу використовується функція **fmt.Printf()**. Вона дозволяє вставляти значення змінних у рядок за допомогою спеціальних специфікаторів (вербаторів).

7. Синтаксис fmt.Printf()

fmt.Printf("Рядок форматування зі специфікаторами", аргументи...)

8. Синтаксис fmt.Scanf()

fmt.Scanf("формат", &змінна)

9. В якому пакеті знаходяться функції введення та виведення?

Усі базові функції введення/виведення (Print, Scan, Printf, Scanf) знаходяться у стандартному пакеті **fmt** (скорочення від *format*).

10. Вивід і введення

Виведення: * fmt.Print() / fmt.Println() — для звичайного виводу (з переходом на новий рядок або без).

fmt.Printf() — для виводу за шаблоном.

Введення: * fmt.Scan() / fmt.Scanln() — для простого зчитування значень.

fmt.Scanf() — для зчитування даних, що відповідають певному формату.